

大数据方向系列课程：大数据实时处理技术

第4章

Spark 运行模式



目录

Main Contents

1.1

Spark架构与运行逻辑

1.2

Spark运行时架构

1.3

Spark应用部署与调度

1.4

Spark集群管理器

1.5

选择合适的集群管理器





本章目标

目标知识点	记忆 remember	理解 understand	应用 apply	分析 analyze	评价 evaluate	创新 create
Spark 架构与运行逻辑	★	★	★	★		
Spark 运行时架构	★	★	★	★		
Spark 应用部署与调度	★	★	★			
Spark 集群管理器	★	★	★	★		

根据BLOOM教育目标分类©



任务驱动

- 理解Spark架构、运行逻辑及作业提交流程。掌握Spark运行时架构的组成及工作原理，掌握Spark应用的部署方式及应用间的调度。了解Spark集群管理器的分类，掌握选择一个合适的集群管理器。

01

Part ONE

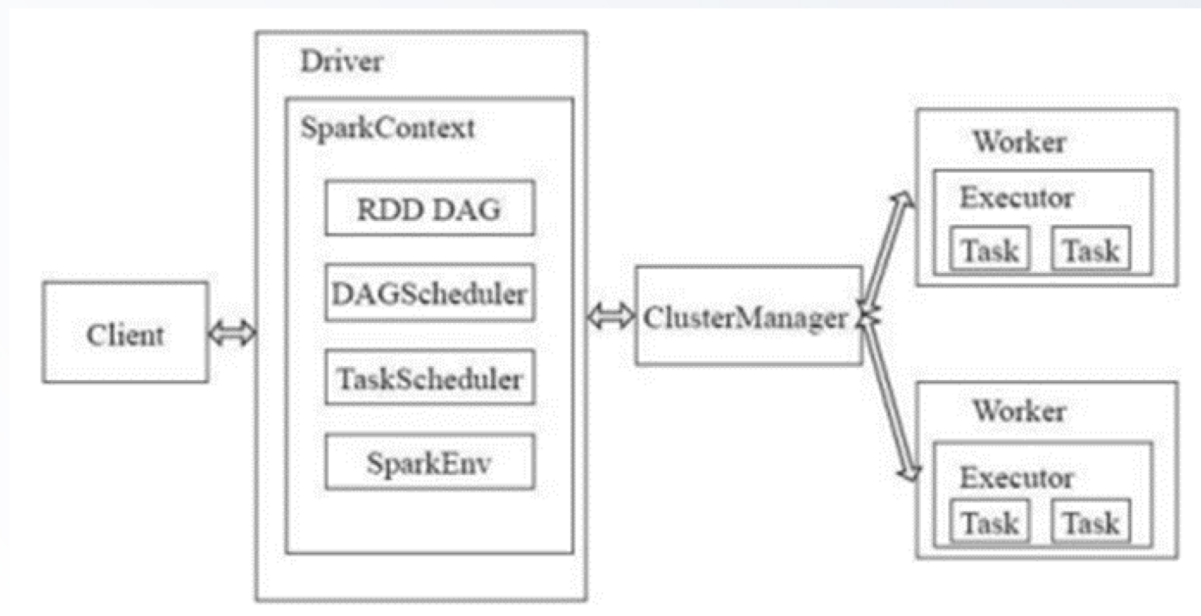
Spark架构组成与运行逻辑

Spark架构组成及基本组件，Spark作业提交流程及作业运行逻辑。



1.1 Spark的架构组成

Spark集群部署后，需要在主节点和从节点分别启动Master进程和Worker进程，对整个集群进行控制。在一个Spark应用的执行过程中，Driver和Worker是两个重要角色。



Driver程序是应用逻辑执行的起点，负责作业的调度，即Task任务的分发，多个Worker用来管理计算节点和创建Executor并行处理任务。在执行阶段，Driver会将Task和Task所依赖的file和jar序列化后传递给对应的Worker机器，同时Exucutor对相应数据分区的任务进行处理。

1.2 Spark的架构中的基本组件

- **Client** : 用户提交作业的客户端。
- **Driver** : 运行Application的main()函数并负责创建SparkContext。
- **SparkContext** : 整个应用的上下文, 控制应用的生命周期。
- **RDD** : 分布式数据集, Spark的基本计算单元, 一组RDD形成执行的有向无环图RDD Graph。
- **DAG Scheduler** : 根据Job构建基于Stage的DAG工作流, 并提交Stage给TaskScheduler。
- **TaskScheduler** : 将Task分发给Executor执行。
- **SparkEnv** : 线程级别的上下文, 存储运行时的重要组件的引用。
- **ClusterManager** : 指的是在集群上获取资源的外部服务, 即运行模式。目前支持至少三种类型: Standalone、Apache Mesos和Hadoop Yarn。
- **Worker** : 集群中任何运行Application代码的工作节点, 运行一个或多个Executor进程。
- **Executor** : 运行在Worker的Task执行器, Executor启动线程池运行Task, 并且负责将数据存在内存或者磁盘上。每个Application都会申请各自的Executor来处理任务。

1.3 Spark的运行逻辑—作业提交流程

Spark在不同运行模式下作业提交流程有所不同，总体来说：

Spark作业 提交流程

1

Client提交应用。

2

Master会找到一个Worker或直接在客户端启动Driver。

3

Driver向Master或者资源管理器（如：YARN）申请资源。

4

Driver将应用转化为RDD有向无环图，再由DAGScheduler将RDD转化为Stage提交给TaskScheduler。

5

TaskScheduler将Stage提交给Executor进行执行。

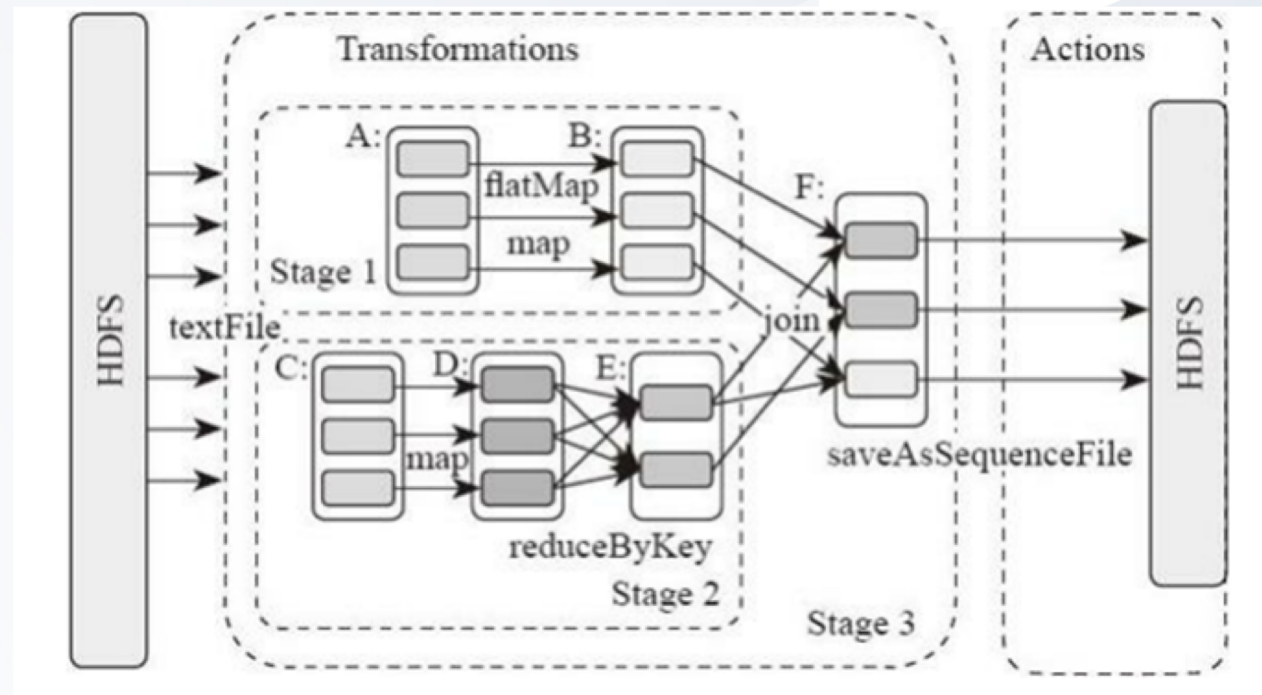
6

任务执行过程中其他组件协同工作，保证整个执行过程。

1.4 Spark的运行逻辑—作业运行逻辑

SparkContext读入文件并创建出一系列的RDD。Spark根据RDD之间不同的依赖关系切分形成不同的阶段（Stage），一个阶段包含一系列函数进行流水线执行。RDD可以有两种操作算子：转换（Transformation）和行动（Action）。

- 在调用行动操作之前，RDD都只是存储着可以让我们计算出具体数据的描述信息。Spark调度器会创建出用于计算行动操作的RDD物理执行计划。
- 要触发实际计算，需要对其调用一个行动操作，如collect()将数据收集到驱动程序中。这时RDD的每个分区都会被物化出来并发送到驱动程序中
- Spark调度器从最终被调用行动操作的RDD触发，向上回溯所有必须计算的RDD。调度器会访问RDD的父节点、父节点的父节点，以此类推，递归向上生成计算所有必要的父RDD的物理计划。



02

Part TWO

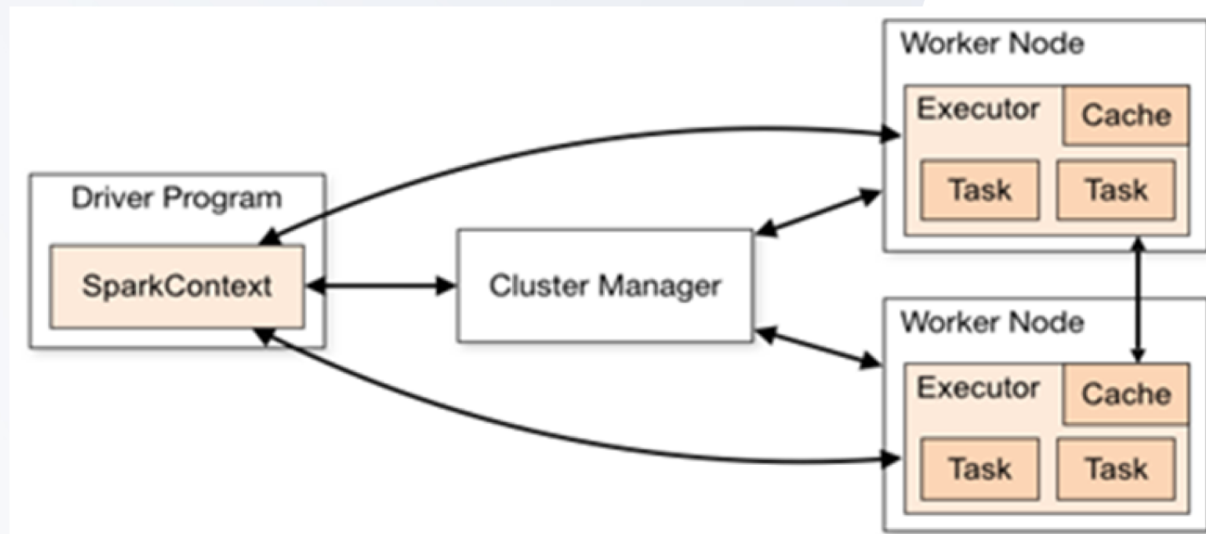
Spark 运行时架构

Spark的驱动器节点 (Driver)、执行器
节点 (Executor)、集群管理器 (Cluster
Manager)



2.1 Spark的运行架构

Spark可以在各种各样的集群管理器（Hadoop YARN、Apache Mesos和Spark自带的独立集群管理器）上运行，所以Spark应用既能够适应专用集群，又能用于共享的云计算环境。



在分布式环境下，Spark集群采用的是主/从结构。在一个Spark集群中，有一个节点负责中央协调，调度各个分布式工作节点。这个中央节点被称为**驱动器（Driver）节点**，与之对应的工作节点被称为**执行器（Executor）节点**。驱动器节点可以和大量的执行器节点进行通信，它们也都作为独立的Java进程运行。驱动器节点和所有的执行器节点一起被称为**一个Spark应用（Application）**。Spark应用通过一个叫作**集群管理器（Cluster Manager）**的外部服务在集群中的机器上启动。Spark自带的集群管理器被称为独立集群管理器。Spark也能运行在Hadoop YARN和Apache Mesos这两大开源集群管理器上。



2.2 Spark的运行时架构—驱动器节点

Spark驱动器是执行程序中的main()方法的进程。它执行用户编写的用来创建SparkContext、创建RDD、以及进行RDD的转化操作和行动操作的代码。当启动Spark shell时，就启动了一个Spark驱动器程序，Spark shell 总是会预先加载一个叫 sc 的SparkContext对象。驱动程序一旦终止，Spark应用也就结束了。

驱动器程序在Spark应用中两个主要职责：

- **将用户程序转为任务**

Spark驱动器程序负责把用户程序转为多个物理执行的单元，也被称为任务（Task）。Spark程序隐式地创建出了一个由操作组成的逻辑上的有向无环图（DAG）。当驱动器程序运行时，会把这个逻辑图转为物理执行计划。

- **为执行器节点调度任务**

有了物理执行计划后，Spark驱动器程序必须在各执行器进程间协调任务的调度。执行器进程启动后，会向驱动器程序注册自己。因此，驱动器程序进程始终对应用中所有的执行器节点有完整的记录。每个执行器节点代表一个能够处理任务和存储RDD数据的进程。

2.3 Spark的运行时架构—执行器节点

Spark执行器节点是一种工作进程，负责在Spark作业中运行任务，任务间相互独立。Spark应用启动时，执行器节点就被同时启动，并且始终伴随整个Spark应用的生命周期。如果有执行器节点发生了异常或崩溃，Spark应用也可以继续执行。

- **执行器进程有两大作用：**

- 负责运行组成Spark应用的任务，并将结果返回给驱动器进程；
- 通过自身的块管理器（Block Manager）为用户程序中要求缓存的RDD提供内存式存储。RDD是直接缓存在执行器进程内的，因此任务可以在运行时充分利用缓存数据加速运算。

2.4 Spark的运行时架构—集群管理器

驱动器节点和执行器节点是如何启动的呢？

Spark依赖于集群管理器来启动执行器节点，而在某些特殊情况下（在YARN的工作节点上，Spark不仅可以运行执行器进程，还可以运行驱动器进程），也依赖集群管理器来启动驱动器节点。集群管理器是Spark中的可插拔式组件。这样，除了Spark自带的独立集群管理器，Spark也可以运行在其他外部的集群管理器上，例如：YARN和Mesos。



1

Standalone独立集群管理器

Spark自带的简单集群管理器，可以轻松设置集群。

2

YARN资源管理器

Hadoop 2.x 中提供的资源管理器。

3

Mesos集群管理器

一个通用的集群管理器，也可以运行MapReduce和服务应用程序。

2.5 Spark的运行时架构 — 启动一个程序

无论使用的是哪一种集群管理器，都可以使用Spark提供的统一脚本spark-submit将Spark应用提交到相应的集群管理器上。通过不同的配置选项，spark-submit可以连接到相应的集群管理器上，并控制应用所使用的资源数量。在使用某些特定集群管理器时，spark-submit也可以将驱动器节点运行在集群内部（如一个YARN的工作节点）。但是对于其他的集群管理器，驱动器节点只能被运行在本地机器上。

spark-submit通过--master参数指定要连接的集群，--master标记可接收的值如下表：

值	说明
spark://host:port	连接到指定端口的Spark独立集群上，默认情况下Spark独立节点使用7077端口
mesos://host:port	连接到指定端口的Mesos集群上，默认情况下Mesos主节点监听5050端口
yarn	连接到一个YARN集群上，需设置环境变量HADOOP_CONF_DIR指向Hadoop配置目录获取集群信息；分两种模式 " --deploy-mode client " 和 " --deploy-mode cluster "
yarn-client	相当--master yarn --deploy-mode client
yarn-cluster	相当--master yarn --deploy-mode cluster
local	运行本地模式，使用单核心
local[N]	运行本地模式，使用N个核心
local[*]	运行本地模式，使用尽可能多的核心

2.6.1 各模式下运行spark自带实例SparkPi

● local模式：

```
# cd $SPARK_HOME
# ./bin/spark-submit
  --class org.apache.spark.examples.SparkPi
  --master local
  ./examples/jars/spark-examples_2.11-2.3.3.jar
```

□ 解释下命令：

- ✓ --class 类名，Spark自带的SparkPi案例
- ✓ --master local 本地模式
- ✓ ./examples/spark-examples_2.11-2.3.3.jar是Spark自带案例，也可以是自己写的Spark程序的jar包

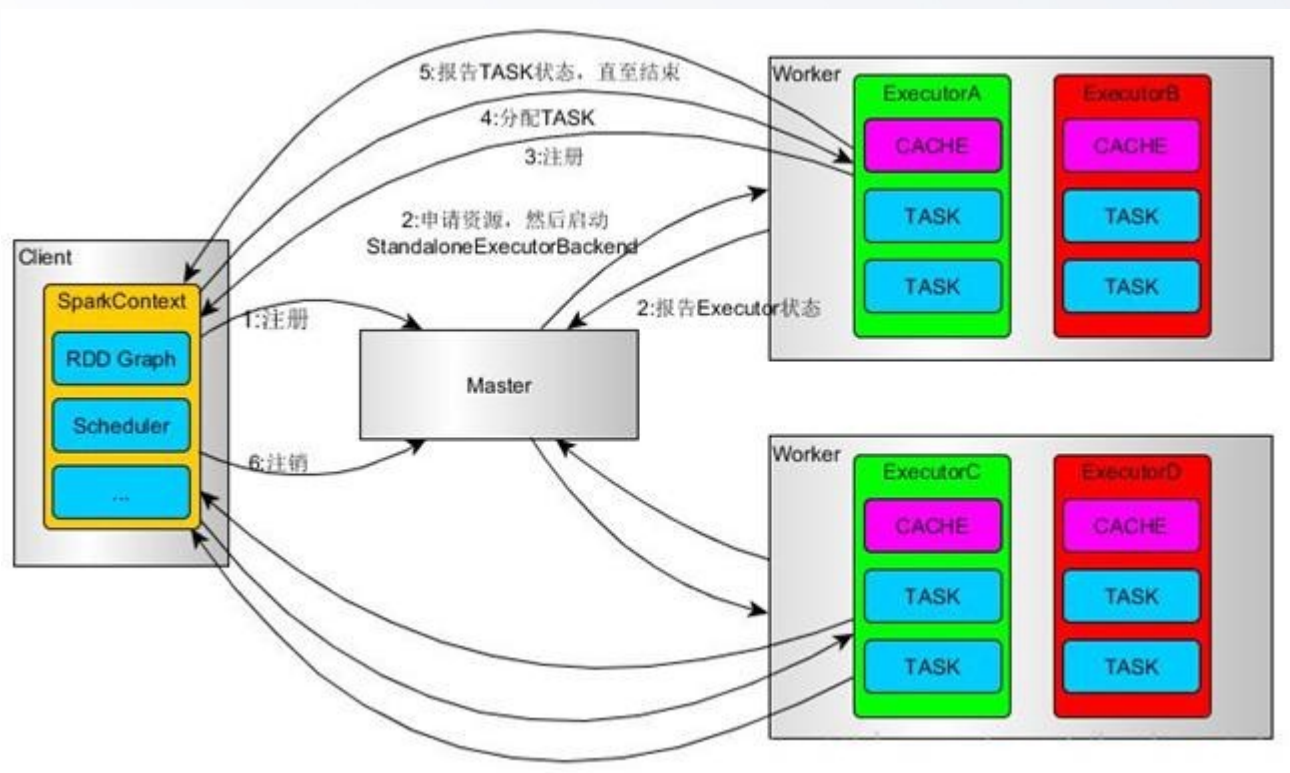
□ 运行结果如下图：

```
(base) [root@master spark-2.3.3-bin-hadoop2.6]# ./bin/spark-submit --class org.apache.spark.examples.SparkPi --master local ./examples/jars/spark-examples_2.11-2.3.3.jar
20/08/31 15:28:50 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
Pi is roughly 3.1441357206786034
```

2.6.2 Spark运行模式

● Spark On Standalone运行原理

Standalone模式是Spark实现的资源调度框架，其主要的节点有Client节点、Master节点和Worker节点。其中Driver可以运行在Master节点上或本地Client端。当用spark-shell交互式工具提交Spark的Job时，Driver在Master节点上运行；当使用spark-submit工具提交Job或者在IDEA等开发上使用new SparkConf.setManager(“spark://master:7077”)方式运行Spark任务时，Driver是运行在本地Client端上的。



2.6.2 各模式下运行spark自带实例SparkPi

● standalone模式：

```
# cd $SPARK_HOME
# ./bin/spark-submit
  --class org.apache.spark.examples.SparkPi
  --master spark://master:7077
  ./examples/jars/spark-examples_2.11-2.3.3.jar
```

□ 解释下命令：

- ✓ --class 类名，Spark自带的SparkPi案例
- ✓ --master spark://master:7077 standalone模式
- ✓ ./examples/spark-examples_2.11-2.3.3.jar是Spark自带案例，也可以是自己写的Spark程序的jar包

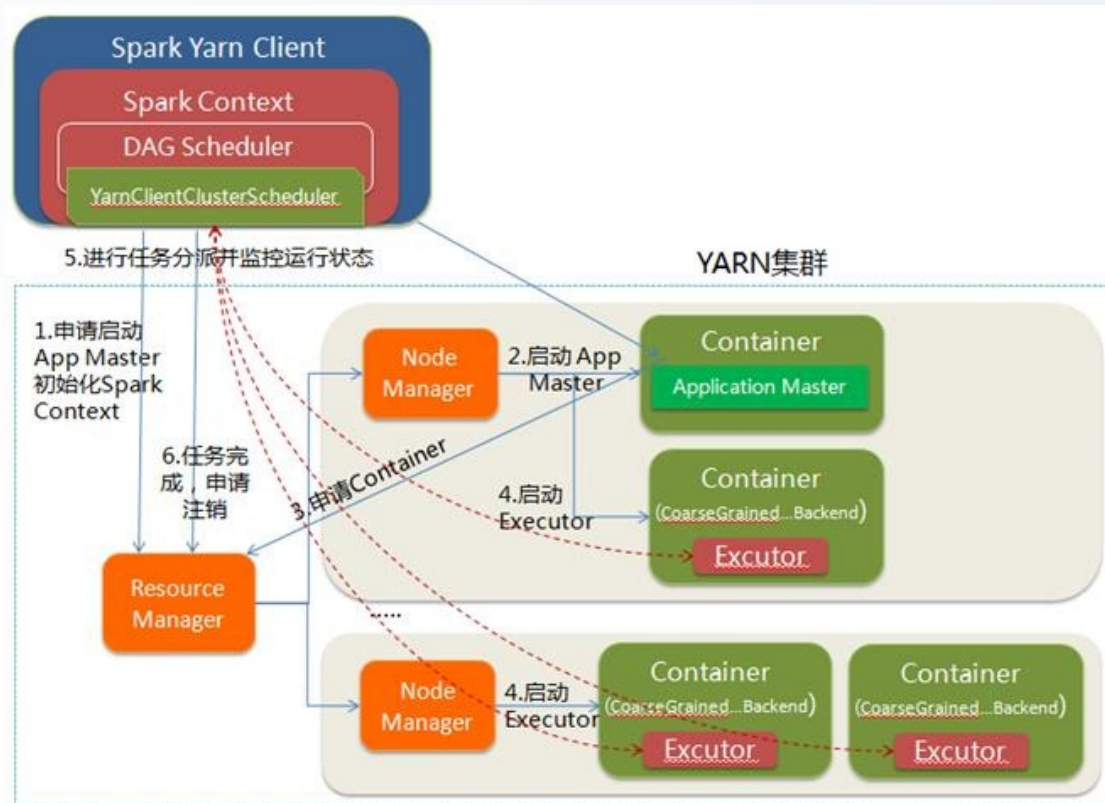
□ 运行结果如下图：

```
(base) [root@master spark-2.3.3-bin-hadoop2.6]# ./bin/spark-submit --class org.apache.spark.examples.SparkPi --master
spark://master:7077 ./examples/jars/spark-examples_2.11-2.3.3.jar
20/08/31 15:52:40 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java
classes where applicable
Pi is roughly 3.1465357326786636
```

2.6.3 Spark运行模式

● YARN-Client运行原理

YARN-Client模式中，Driver在客户端本地运行，这种模式可以使得Spark Application和客户端进行交互，所以可以通过WebUI访问Driver的状态，默认是http://hadoop1:4040访问，而YARN通过http://hadoop1:8088访问。



2.6.3 各模式下运行spark自带实例SparkPi

● yarn-client模式：

```
# cd $SPARK_HOME
# ./bin/spark-submit
  --class org.apache.spark.examples.SparkPi
  --master yarn --deploy-mode client
  ./examples/jars/spark-examples_2.11-2.3.3.jar
```

□ 解释下命令：

- ✓ --class 类名，Spark自带的SparkPi案例
- ✓ --master yarn --deploy-mode client yarn-client模式
- ✓ ./examples/spark-examples_2.11-2.3.3.jar是Spark自带案例，也可以是自己写的Spark程序的jar包

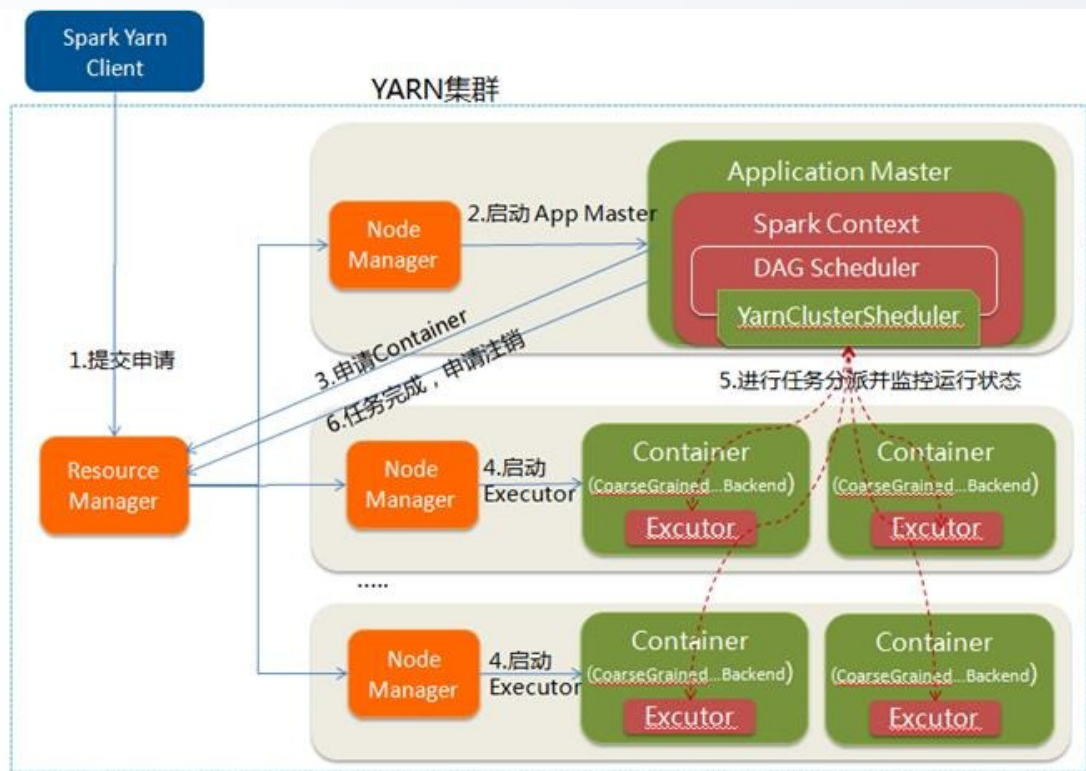
□ 运行结果如下图：

```
(base) [root@master spark-2.3.3-bin-hadoop2.6]# ./bin/spark-submit --class org.apache.spark.examples.SparkPi --master yarn --deploy-mode client ./examples/jars/spark-examples_2.11-2.3.3.jar
20/08/31 15:58:34 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
20/08/31 15:58:38 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling back to uploading libraries under SPARK_HOME.
20/08/31 15:59:21 WARN YarnScheduler: Initial job has not accepted any resources; check your cluster UI to ensure that workers are registered and have sufficient resources
Pi is roughly 3.145435727178636
```

2.6.4 Spark运行模式

● YARN-Cluster运行原理

在YARN-Cluster模式中，YARN将用户提交的应用分两个阶段运行该应用程序：第一个阶段是把Spark的Driver作为一个ApplicationMaster在YARN集群中先启动；第二个阶段是由ApplicationMaster创建应用程序，然后为它向ResourceManager申请资源，并启动Executor来运行Task，同时监控它的整个运行过程，直到运行完成。



2.6.4 各模式下运行spark自带实例SparkPi

● yarn-cluster模式：

```
# cd $SPARK_HOME
# ./bin/spark-submit
  --class org.apache.spark.examples.SparkPi
  --master yarn --deploy-mode cluster
  ./examples/jars/spark-examples_2.11-2.3.3.jar
```

□ 解释下命令：

- ✓ --class 类名，Spark自带的SparkPi案例
- ✓ --master yarn --deploy-mode cluster yarn-cluster模式
- ✓ ./examples/spark-examples_2.11-2.3.3.jar是Spark自带案例，也可以是自己写的Spark程序的jar包

□ 运行结果如下图：

```
(base) [root@master spark-2.3.3-bin-hadoop2.6]# ./bin/spark-submit --class org.apache.spark.examples.SparkPi --master yarn --deploy-mode cluster ./examples/jars/spark-examples_2.11-2.3.3.jar
20/08/31 16:03:36 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable
20/08/31 16:03:38 WARN Client: Neither spark.yarn.jars nor spark.yarn.archive is set, falling back to uploading libraries under SPARK_HOME.
```

2.7 小结

- 回顾一下在集群上运行Spark应用的过程，可以将本节的主要概念作如下总结：
 - 用户通过spark-submit脚本提交应用
 - spark-submit脚本启动驱动器程序，调用用户定义的main()方法
 - 驱动器程序与集群管理器通信，申请资源以启动执行器节点
 - 集群管理器为驱动器程序启动执行器节点
 - 驱动器进程执行用户应用中的操作。根据程序中所定义的对RDD的转化操作和行动操作，驱动器节点把工作以任务的形式发送到执行器进程
 - 任务在执行器程序中进行计算并保存结果
 - 如果驱动器程序的main()方法退出，或者调用了SparkContext.stop()，驱动器程序会终止执行器进程，并且通过集群管理器释放资源

03

Part THREE

Spark应用部署与调度

使用spark-submit部署应用，打包代码与依赖，
Spark应用内与应用间调度





3.1 使用spark-submit部署应用

Spark为各种集群管理器提供了统一的工具spark-submit用来提交作业。 spark-submit的一般格式：

```
# bin/spark-submit [options] <app jar | python file> [app options]
```

其中：

- [options]是传给spark-submit的参数列表。可以运行bin/spark-submit --help列出所有可以接收的参数。
- <app jar | python file>表示包含应用入口的JAR包或Python脚本。
- [app options]是传给应用的参数选项。

参数	说明
--master	表示要连接的集群管理器，这个参数可接收的选项可参见上一节的表格
--deploy-mode	选择驱动器程序在本地(client)启动，还是在集群中一个工作节点(cluster)上启动
--class	运行Java或Scala程序时应用的主类
--name	应用的显示名称，会显示在Spark的网页用户界面中
--jars	需要上传并放到应用的CLASSPATH中的JAR包的列表，如依赖的第三方JAR包
--files	需要放到应用工作目录中的文件的列表，一般用来设置要分发到各节点的数据文件
--py-files	需要添加到PYTHONPATH中的文件的列表，可以是.py、.egg以及.zip等文件
--executor-memory	执行器进程使用的内存量，以字节为单位，可以使用后缀指定更大的单位，如“15g”
--driver-memory	驱动器进程使用的内存量，以字节为单位，可以使用后缀指定更大的单位，如“15g”

3.2 使用spark-submit部署应用—例子

- 使用独立集群模式提交Java应用

```
# ./bin/spark-submit --master spark://hostname:7077 --deploy-mode cluster \  
--class com.databricks.examples.SparkExample --name " Example Program " \  
--jars dep1.jar,dep2.jar,dep3.jar --total-executor-cores 300 \  
--executor-memory 10g myApp.jar
```

- 使用YARN客户端模式提交Python应用

```
# export HADOOP_CONF_DIR=/opt/hadoop/conf  
# ./bin/spark-submit --master yarn --deploy-mode client \  
main.py --name " Example Program " \  
--py-files somelib-1.2.egg,otherlib-4.4.zip,other-file.py
```

3.3 打包代码与依赖

在大多数情况下用户程序需要依赖第三方的库。如果用户程序引入了任何既不在org.apache.spark包内也不属于语言运行时的库的依赖，这时就需要确保所有的依赖在该Spark应用运行的时候都能被找到。

- **对于Python用户**

可以通过标准的Python包管理器pip直接在集群中的所有机器上安装所依赖的库，或者把依赖手动安装到Python安装目录下的site-packages/目录中。也可以使用spark-submit的 --py-Files参数提交独立的库。

- **对于Java和Scala用户**

可以通过spark-submit的 --jars参数提交独立的JAR包依赖。但是一般Java和Scala的工程会依赖很多库，并且库可能还要依赖其他的库，当你向Spark提交应用时，必须把应用的整个依赖传递给应用，这时使用手动维护和提交依赖的JAR包是很繁琐且不方便的。常规的做法是使用构建工具，生成单个大JAR包，包含应用的所有的传递依赖。这通常被称为超级JAR或者组合JAR，大多数Java或Scala的构建工具都支持生成这样的工件。

3.4.1 打包代码与依赖—例子

Java和Scala中使用最广泛的构建工具是Maven和SBT。它们都可以用于这两种语言，不过通常Maven用于Java工程，而SBT则一般用于Scala工程。

1、使用Maven构建的用Java编写的Spark应用

Maven提供了pom.xml文件，其中包含本次构建的定义。Maven中默认用户代码在工程根目录下的src/main/java目录中，pom.xml在工程根目录下。

案例：[使用Maven构建Spark应用。](#)

2、使用SBT构建的用Scala编写的Spark应用

SBT是一个通常在Scala工程中使用的比较新的构建工具。SBT预期的目录结构和Maven相似。在工程的根目录中，创建出一个叫build.sbt的构建文件，源代码放在src/main/scala中。SBT构建文件是用配置语言写成的，所以可能需要提前读一读最新的文档，以了解构建文件格式上的改变。

案例：[使用SBT构建Spark应用。](#)

3.4.2 打包代码与依赖—例子

3、依赖冲突

当用户应用与Spark本身依赖同一个库时可能会发生依赖冲突，导致程序崩溃。依赖冲突表现为Spark作业执行过程中抛出NoSuchMethodError、ClassNotFoundException，或其他与类加载相关的JVM异常。

- **对于这种问题，主要有两种解决方案：**

- 修改应用，使其使用的依赖库的版本与Spark所使用的相同
- 使用被称为“shading”的方式打包应用。shading可以让你以另一个命名空间保留冲突的包，并自动重写应用的代码使得它们使用重命名后的版本。

3.5 Spark应用内与应用间调度

在实际生产环境中，不会只有一个用户向集群提交作业。集群通常是在多个用户间共享的。共享的环境会带来调度方面的挑战：如果两个用户都启动了希望使用整个集群所有资源的应用，该如何处理？Spark有一系列调度策略来保障资源不会被过度使用，还允许工作负载设置优先级。

- Spark主要依赖集群管理器来在Spark应用间共享资源。当Spark应用向集群管理器申请执行器节点时，应用收到的执行器节点个数可能比它申请的更多或更少。大多数集群管理器支持队列，可以为队列定义不同优先级或容量限制，这样Spark就可以把作业提交到相应的队列中。
- Spark应用有一种特殊情况，就是“长期运行”的应用。这些应用不主动退出，如Spark SQL中的JDBC服务器就是一个长期运行的Spark应用。由于这个应用本身就是为多用户调度工作的，所以它需要一种细粒度的调度机制来强制共享资源。Spark提供了一种用来配置应用内调度策略的机制。Spark内部的公平调度器会让长期运行的应用定义调度任务的优先级队列。

04

Part FOUR

Spark集群管理器

独立集群管理器，Hadoop YARN集群管理器，Apache Mesos集群管理器



4.1 独立集群管理器

Spark独立集群管理器提供在集群上运行应用的简单方法。这种集群管理器由一个主节点和多个工作节点组成，各自都分配有一定量的内存和CPU核心。当提交应用时，可以配置执行器进程使用的内存量，以及所有执行器进程使用的CPU核心总数。

● 启动独立集群管理器

要使用独立集群管理器，请按如下步骤启动集群：

- 将编译好的Spark复制到所有机器的一个相同的目录下，如：`/home/yourname/spark`
- 设置从主节点机器到其他机器的SSH免密码登陆
- 编辑主节点的`conf/slaves`文件并填写所有工作节点的主机名
- 在主节点上运行`sbin/start-all.sh`以启动集群
- 要停止集群，需要在主节点上运行`sbin/stop-all.sh`
- 如果想手动启动集群，可以使用Spark的`bin/`目录下的`spark-class`脚本分别手动启动主节点和工作节点：在主节点上输入`# bin/spark-class org.apache.spark.deploy.master.Master`；然后在工作节点上输入`#bin/spark-class org.apache.spark.deploy.worker.Worker spark://masternode:7077`

4.2 独立集群管理器—提交应用

● 提交应用

要向独立集群管理器提交应用，需在主节点上将spark://masternode: 7077作为参数传给spark-submit命令：

```
# spark-submit --master spark://masternode:7077 myapp
```

也可以使用 --master 参数以同样的方式启动spark-shell或pyspark，来连接到集群：

```
# pyspark --master spark://masternode:7077
```

独立集群管理器支持两种部署模式：

■ 客户端模式(client)

驱动器程序会运行在执行spark-submit的机器上，是spark-submit命令的一部分。可以直接看到驱动器程序的输出，也可以直接输入数据进去，但是这要求提交应用的机器与工作节点间有很快的网络速度，并且在程序运行的过程中始终可用。

■ 集群模式(cluster)

驱动器程序会作为某个工作节点上一个独立的进程运行在独立集群管理器内部。它也会连接主节点来申请执行器节点。在这种模式下，spark-submit是“一劳永逸”型，即可以在应用运行后关掉客户端电脑。

4.3 独立集群管理器—配置资源用量

● 配置资源用量

如果在多应用间共享Spark集群，需要决定如何在执行器进程间分配资源。独立集群管理器使用基础的调度策略，这种策略允许限制各个应用的用量来让多个应用并发执行。Apache Mesos支持应用运行时的动态资源共享，而YARN则有分级队列的概念，可以让你限制不同类别的应用的用量。

在独立集群管理器中，资源分配靠下面两个设置来控制。

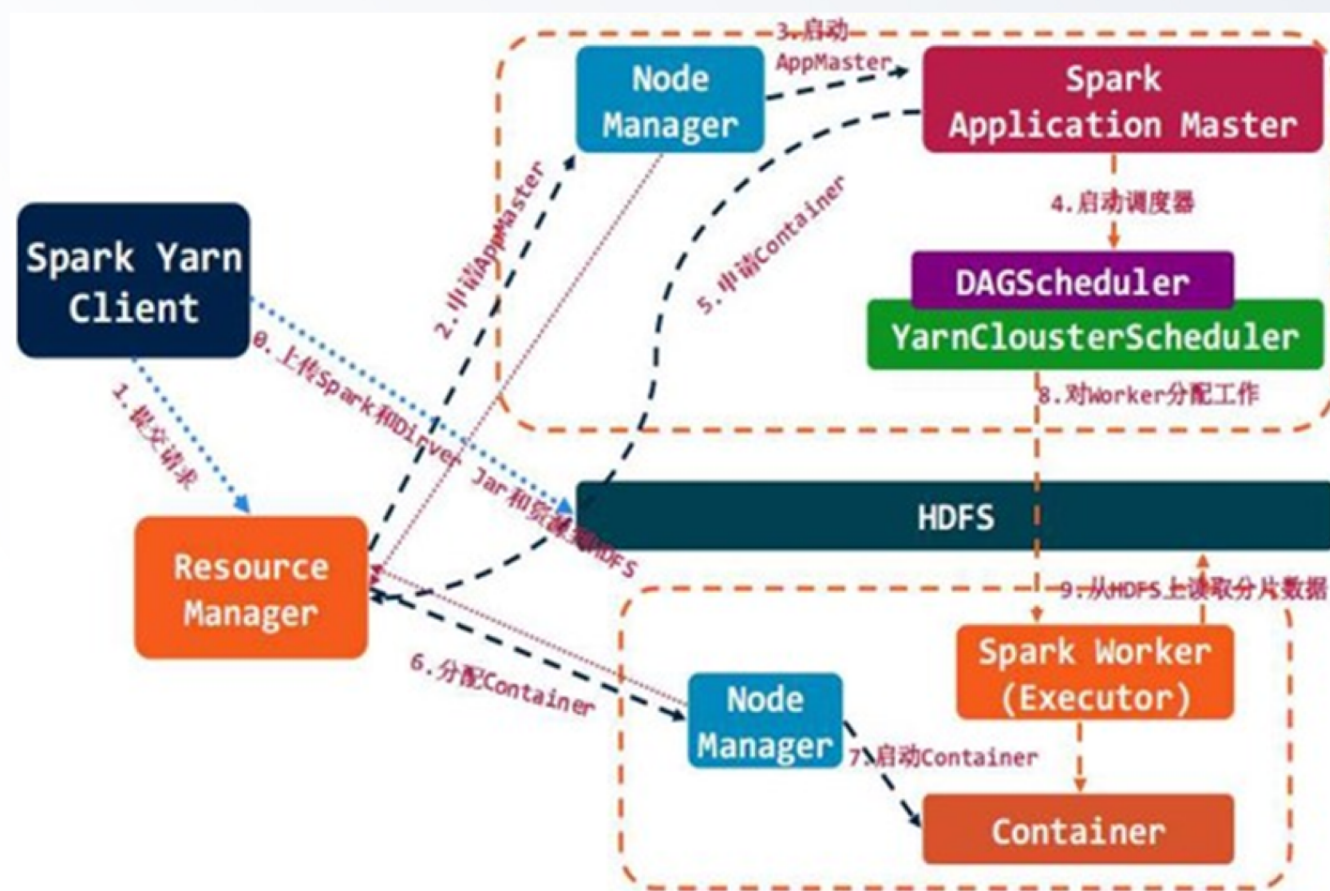
- 执行器进程内存：通过spark-submit的 `--executor-memory` 参数来配置此项。此项的默认值是1GB。
- 占用核心总数的最大值：通过spark-submit的 `--total-executor-cores` 参数来配置此项。或在Spark配置文件中设置 `spark.cores.max` 的值。此项的默认值是无限。

● 高度可用性

在生产环境中运行时，独立模式也能很好地支持工作节点的故障。如果想让集群的主节点也拥有高可用性，Spark支持使用Apache ZooKeeper来维护多个备用的主节点，并在一个主节点失败时切换到新的主节点上。可参见《搭建Spark HA集群操作手册》。

4.4 Hadoop YARN 集群管理器

YARN是在Hadoop2.0中引入的集群管理器，它可以让多种数据处理框架运行在一个共享的资源池上，并且通常安装在与Hadoop文件系统（HDFS）相同的物理节点上。在YARN集群上运行Spark是很有意义的，可以让Spark在存储数据的物理节点上运行，以快速访问HDFS中的数据。



4.5 Hadoop YARN 集群管理器—提交应用

在Spark中使用YARN很简单：只需要设置指向Hadoop配置目录的环境变量，然后使用spark-submit向一个特殊的主节点URL提交作业即可。可以找到Hadoop的配置目录，并设置为环境HADOOP_CONF_DIR，这个目录包含yarn-site.xml和其他配置文件。

例如：可以通过如下方式提交应用。

```
export HADOOP_CONF_DIR="/etc/hadoop/conf"  
spark-submit --master yarn myapp
```

- **和独立集群管理器一样，Spark on YARN也有两种将应用连接到集群的模式：**

- 客户端模式（client）：驱动器程序运行在提交应用的机器上。
- 集群模式（cluster）：驱动器程序也运行在一个YARN容器内部。

使用spark-submit的 --deploy-mode参数设置不同的模式。

Spark的交互式shell以及pyspark也都可以运行在YARN上。只要设置好HADOOP_CONF_DIR并使用--master yarn参数即可。注意，由于这些应用需要从用户处获取输入，所以只能运行于客户端模式下。

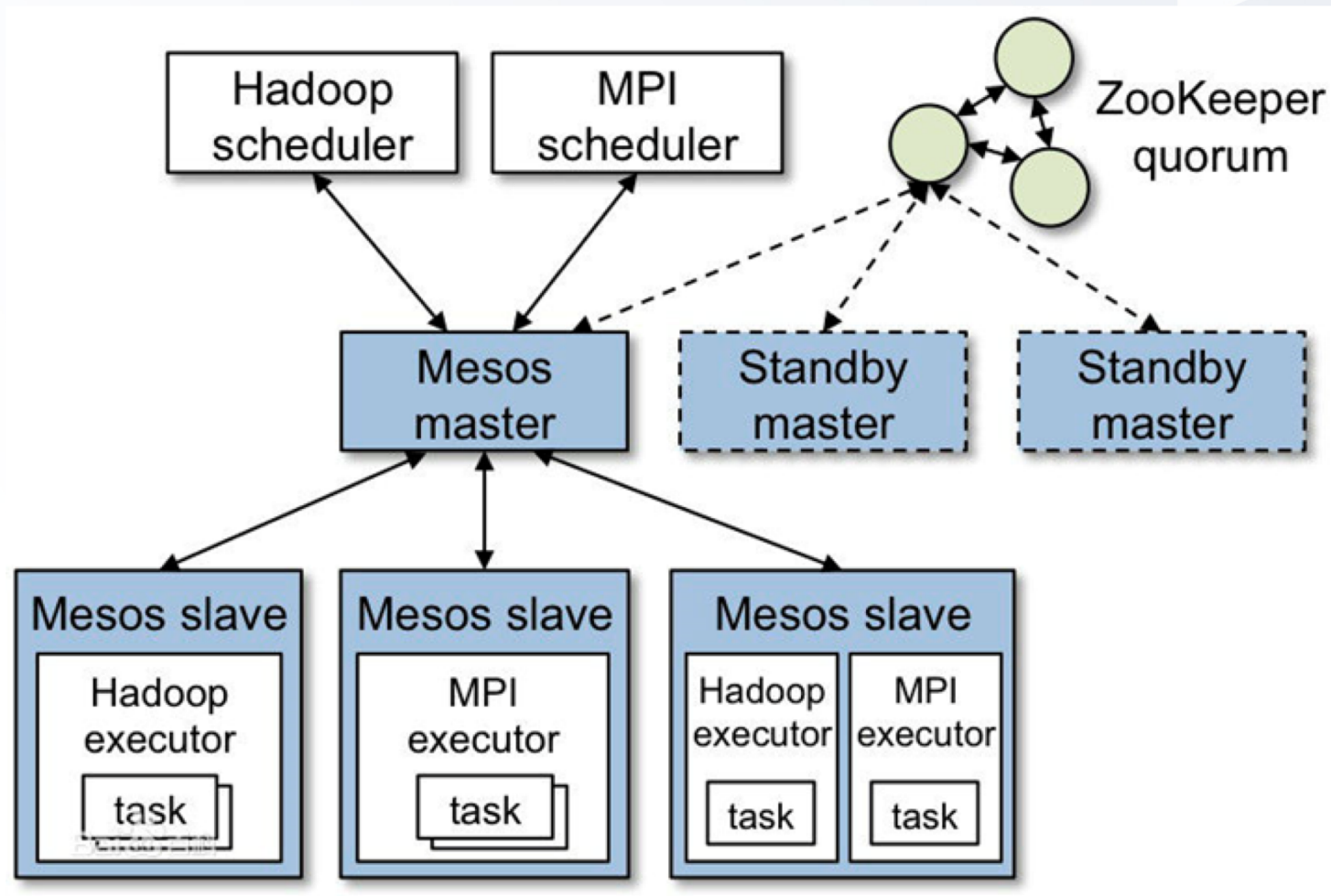
4.6 Hadoop YARN 集群管理器—配置资源用量

当Spark在YARN上运行时，根据在spark-submit或spark-shell等脚本的 `--num-executors` 参数中设置的值，Spark应用会使用固定数量的执行器节点。默认情况下值为2。可以通过 `--executor-memory` 设置每个执行器的内存用量，通过 `--executor-cores` 设置每个执行器进程从YARN中占用的核心数目。对于给定的硬件资源，Spark通常在用量较大而总数较少的执行器组合上表现得更好，因为这样Spark可以优化各执行器进程间的通信。

需要注意的是，一些集群限制了每个执行器进程的最大内存（默认为8GB），不允许使用更大的执行器进程。出于资源管理的目的，某些YARN集群被设置为将应用调度到多个队列中。使用 `--queue` 选项来选择队列的名字。

4.7.1 Apache Mesos 集群管理器

Apache Mesos是一个通用集群管理器，既可以运行分析型工作负载，又可以运行长期运行的服务（如网页服务或键值对存储服务）。



4.7.2 Apache Mesos 集群管理器

● Mesos优点：

- 资源管理策略Dominant Resource Fairness(DRF), 这是Mesos的核心，也是我们把Mesos比作分布式系统Kernel的根本原因。通俗讲，Mesos能够保证集群内的所有用户有平等的机会使用集群内的资源，这里的资源包括CPU，内存，磁盘等等。
- 轻量级。相对于 YARN，Mesos 只负责offer资源给framework，不负责调度资源。这样，理论上，我们可以让各种东西使用Mesos集群资源，而不像YARN只拘泥于Hadoop，我们需要做的是开发调度器（mesos framework）。
- 提高分布式集群的资源利用率：这是一个 generic 的优点。其实所有的集群管理工具都是为了提高资源利用率。VM的出现，催生了IaaS；容器的出现，催生了K8s, Mesos等等。简单讲，同样多的资源，我们利用IaaS把它们拆成VM 与 利用K8s/Mesos把它们拆成容器，显然后者的资源利用率更高。

● Mesos缺点：

- Mesos项目还在紧锣密鼓的开发中，很多功能还不完善。譬如，集群资源抢占还不支持。



4.8 Apache Mesos 集群管理器—提交应用

要在Mesos上使用Spark，需要将一个 `mesos://...` 的URI传给spark-submit，例如：

```
# spark-submit --master mesos://masternode:5050 myapp
```

在运行多主节点时，可以使用ZooKeeper来为Mesos集群选出一个主节点。这时，传给spark-submit的URI应用是 `mesos://zk://...` 指向一个ZooKeeper节点列表，假设有三个ZooKeeper节点(node1、node2、node3)，并且ZooKeeper分别运行在三台机器的2181端口上时，应该使用如下的命令提交作业：

```
# spark-submit --master mesos://zk://node1:2181/mesos,node2:2181/mesos,node3:2181/mesos myapp
```

客户端和集群模式

Spark1.2中，在Mesos上Spark只支持以客户端的部署模式运行应用。也就是说，驱动器程序必须运行在提交应用的那台机器上。如果希望在Mesos集群中运行驱动器节点，可以使用Aurora或Chronos框架。

4.9 Apache Mesos 集群管理器—调度模式

- Mesos提供了粗粒度(coarse-grained)和细粒度(fine-grained)两种运行模式。

- **细粒度模式**：是Spark默认模式，这种模式支持资源的抢占，Spark和其他Frameworks以非常细粒度的运行在同一个集群中，每个Application可以根据任务运行的情况在运行过程中动态的获得更多或更少的资源。但是这会在每个task启动的时候增加额外的开销。不适合于一些低延时场景例如交互式查询或者web服务请求等。

- ✓ 优点：启动spark-shell，启动时不占有资源，需要运行task后才去申请。

- ✓ 缺点：Spark中运行的每个task的运行都需要去申请资源，也就是说启动每个task都增加了额外的开销。在一些task数量很多，可是任务量比较轻的应用中，该开销会被放大。例如：遍历一个hdfs中拥有3w分区的数据（56亿条）任务，粗粒度模式耗时：50s；细粒度模式耗时：420s。

- **粗粒度模式**：Spark提前为每个执行器进程分配固定数量的CPU数目，且在应用结束前不会释放这些资源。

可通过向spark-submit传递 `--conf spark.mesos.coarse=true`来打开粗粒度模式。

在实际生产环境中可以在一个Mesos集群中使用混合的调度模式（如：一部分Spark应用的spark.mesos.coarse设置为true，而另一部分不设置）。



4.10 Apache Mesos 集群管理器—配置资源用量

可以通过spark-submit的两个参数 `--executor-memory`和 `total-executor-cores`来控制运行在Mesos上的资源用量。默认情况下Spark会使用尽可能多的核心启动各个执行器节点，来将应用合并到尽量少的执行器实例中，并为应用分配所需要的核心数。如果不设置`total-executor-cores`参数，Mesos会尝试使用集群中所有可用的核心。

05

Part FIVE

选择合适的集群管理器

Spark集群的硬件选择，Spark选择合适的
集群管理器



5.1 Spark集群的硬件选择

如何为Spark配置硬件? 正确的硬件配置取决于使用的场景，接下来给出一些建议：

● 存储系统

由于大多数Spark作业都很可能需要从外部存储系统（如HDFS或HBase）读取输入的数据，所以部署Spark时尽可能靠近这些系统是很重要的。建议如下：

- 尽量在与HDFS相同的节点上运行Spark，简单的方法是在相同节点上设置Spark独立模式集群，并且配置Spark和Hadoop的内存和CPU的使用以避免干扰(Hadoop设置每个任务内存大小的选项`mapred.child.java.opts`以及设置任务数量的选项`mapred.tasktracker.map.tasks.maximum`和`mapred.tasktracker.reduce.tasks.maximum`)。当然也可以在集群管理器(比如Mesos或者Hadoop YARN)上运行Hadoop和Spark。
- 如果不可在相同的节点上运行，建议在与HDFS相同局域网中不同节点上运行Spark。
- 对于像HBase这样的低延时数据存储系统，在与这些存储系统不同的节点上运行计算作业来说，可能更有利于避免干扰。

5.2 Spark集群的硬件选择

● 内存

- **一般来说**，Spark可以在8GB到数百GB内存的任何机器上正常运行。在多数情况下我们建议只为Spark分配最多75%的内存；其余部分供操作系统和缓存区高速缓存存储器使用。
- **需要多少内存取决于应用程序**。如果需要确定的应用程序中某个特定数据集占用内存的大小，可以把这个数据集加载到一个Spark RDD中，然后在Spark监控UI页面(<http://<driver-node>:4040>)中的Storage选项卡下查看它在内存中的大小。需要注意的是，存储级别和序列化格式对内存使用量有很大的影响。如何减少内存使用量的建议，请参阅《Spark调优手册》。
- **最后，需要注意的是Java虚拟机在超过200GB的RAM时表现得并不好**。如果机器有比这更多的RAM，可以在每个节点上运行多个Worker的JVM实例。在Spark的独立模式下,你可以通过conf/spark-env.sh中的SPARK_WORKER_INSTANCES和SPARK_WORKER_CORES两个参数来分别设置每个节点的Worker数量和每个Worker使用的核心数量。

5.3 Spark集群的硬件选择

● 磁盘

尽管Spark可以在内存中执行大量计算，但是仍然需要使用本地磁盘来存储不适合内存的数据以及各阶段的中间结果。建议每个节点配置4-8个磁盘，并且不使用RAID配置（只作为单独挂载点）。在Linux中使用noatime选项挂载磁盘以减少不必要的写入。在Spark中，配置spark.local.dir变量为逗号分隔的本地磁盘列表。如果正在使用HDFS，可以写与HDFS相同的磁盘。

● 网络

根据经验，当数据在内存中时，很多Spark应用程序跟网络有密切的关系。使用10千兆位以太网或者更快的网络是让这些程序变快的最佳方式。这对于“分布式计算”类的应用程序来说尤其如此，例如group-by、reduce-by和SQL join等。

● 处理器

由于Spark实行线程之间的最小共享，所以Spark可以很好地在每台机器上扩展数十个CPU核心。建议为每台机器至少配置8-16个核心。根据工作负载的CPU成本，你可能还需要更多：当数据都在内存中时，大多数应用程序就只跟CPU或者网络有关了。

5.4 Spark选择合适的集群管理器

- **Spark支持多种集群管理器。如何权衡各种集群管理器，下面给出一些推荐准则：**
 - 开始学习Spark时，可以选择独立集群管理器。独立模式安装简单，而且如果只是使用Spark的话，独立集群管理器提供与其他集群管理器完全一样的全部功能。
 - 如果在使用Spark的同时使用其他应用，或者要用到更丰富的资源调度功能，那么YARN和Mesos都能满足需求。大多数Hadoop发行版，一般都预装了YARN。
 - Mesos相对于YARN和独立模式的优点在于其细粒度共享的选项，该选项可以将类似Spark shell这样的交互式应用中的不同命令分配到不同的CPU上。因此这对于多用户同时运行交互式shell的用例更有用处。
 - 在任何时候，最好把Spark运行在运行HDFS的节点上，这样能快速访问存储。可以自行在同样的节点上安装Mesos或独立集群管理器。如果使用YARN，大多数发行版已经把YARN和HDFS安装在一起。
 - 集群管理器处于快速发展中，Spark可能会增加对新的集群管理器的支持。可以随时关注Spark官方文档（ <http://spark.apache.org/docs/latest/> ）来了解最新的选项。

总结、作业

总结



- 1、Spark 架构与运行逻辑
- 2、Spark 运行时架构
- 3、Spark 应用部署与调度
- 4、Spark 集群管理器
- 5、选择合适的集群管理器

作业



疑问



- 1、如何选择一个合适的集群管理器。

Thanks

